Лабораторна робота №1

Паралельний алгоритм решета Ератосфена
на платформі PARCS-Python у Google Cloud

Горобійченко Андрій, група ТК-42
Факультет комп'ютерних наук та кібернетики
Київський національний університет імені Тараса Шевченка

2026

1. Постановка задачі

Знайти всі прості числа у діапазоні $[2, N]$ двома способами:

- **послідовне** решето Ератосфена — як baseline;
- **паралельне** сегментоване решето на платформі **PARCS-Python** у хмарному середовищі Google Cloud Platform.

Порівняти час виконання для різних значень N та різної кількості воркерів K, обчислити speedup і efficiency.

2. Алгоритми

2.1. Послідовне решето Ератосфена

Класичний алгоритм $O(N \log \log N)$: ведемо бітовий масив `is_prime[0..N]`, викреслюємо кратні кожному знайденому простому. Реалізація — файл `sieve_serial.py`.

2.2. Паралельне сегментоване решето

Ідея — розподілити роботу між K воркерами:

1. **Майстер** обчислює всі прості до $\sqrt{N}$ класичним решетом (швидко, бо список малий).
2. Діапазон $(\sqrt{N}, N]$ ділиться на K рівних сегментів між воркерами.
3. Кожен **воркер** отримує свій сегмент $[low, high]$ та список малих простих, викреслює композитні числа і повертає список простих у сегменті.
4. Майстер об'єднує: малі прості + результати всіх воркерів.

Реалізація — `sieve_parallel.py` (клас `Solver`).

3. Розгортання у хмарі

Для експериментів розгорнуто кластер із 9 віртуальних машин у Google Cloud Platform, зона `us-central1-a`:

- **master** — тип `e2-standard-8` (8 vCPU, 32 GB RAM), Container-Optimized OS, Docker-контейнер `hummer12007/parcs-node` зі змінною `PARCS_ARGS=master`;
- **worker1–worker8** — тип `e2-small` (2 vCPU, 2 GB RAM), той самий Docker-образ зі змінною `PARCS_ARGS="worker 10.128.0.2"`.

Firewall-правило `parcs-allow-all` відкриває трафік до VMs на всіх TCP-портах з будь-якої адреси. Воркери підключаються до майстра за внутрішньою IP-адресою `10.128.0.2`.

4. Результати експериментів

Експерименти проводились на VM `master` (`e2-standard-8`, 8 vCPU) з використанням Python 3.11 у Docker-контейнері. Паралелізм реалізовано через `multiprocessing.Pool` з K воркерами. Виміри автоматизовано скриптом `benchmark.py`.

Табл. 1: Час виконання та speedup для різних N і K

N	K	Час (с)	Speedup	Efficiency
10^7	послідовне	1.081	1.00	—
10^7	1	1.369	0.79	0.79
10^7	2	0.725	1.49	0.75
10^7	4	0.382	2.83	0.71
10^7	8	0.365	2.96	0.37
$5 \cdot 10^7$	послідовне	5.373	1.00	—
$5 \cdot 10^7$	1	7.512	0.72	0.72
$5 \cdot 10^7$	2	3.824	1.40	0.70
$5 \cdot 10^7$	4	1.971	2.73	0.68
$5 \cdot 10^7$	8	1.884	2.85	0.36
10^8	послідовне	10.937	1.00	—
10^8	1	14.434	0.76	0.76
10^8	2	7.922	1.38	0.69
10^8	4	4.064	2.69	0.67
10^8	8	3.717	2.94	0.37

4.1. Графік speedup

5. Висновки

1. Паралельний segmented sieve реалізовано на платформі PARCS-Python і розгорнуто у Google Cloud Platform (9 VM: master + 8 воркерів).

2. Експерименти показали, що speedup монотонно зростає з K до $\approx 3\times$ при $K = 8$ на 8-ядерному master-вузлі.

3. При $K = 1$ speedup менший за 1 (0.72–0.79) через накладні витрати `multiprocessing` — створення процесу, серіалізація, об'єднання результатів перевищує виграш від розпаралелювання для одного воркера.

4. При $K = 8$ speedup не досягає теоретичного ідеалу ($8\times$) через: (a) spawning overhead на дрібних задачах, (b) GIL та I/O у Python, (c) неоптимальний розподіл нерівномірної роботи (у верхніх сегментах кандидатів менше).

5. Efficiency (speedup/K) падає з 0.75 при $K = 2$ до 0.37 при $K = 8$ — класична сублінійна масштабованість.

6. Чим більший N, тим стабільніший виграш від паралелізму — накладні витрати amortizуються.

Посилання

- Код проєкту: https://github.com/RedlineKNU/CPLab1
- PARCS-Python: https://github.com/Hummer12007/parcs-python
- Методичка курсу: https://mi.csc.knu.ua/wp-content/uploads/2025/09/Task_1_2_Cloud_Computing_0925.pdf

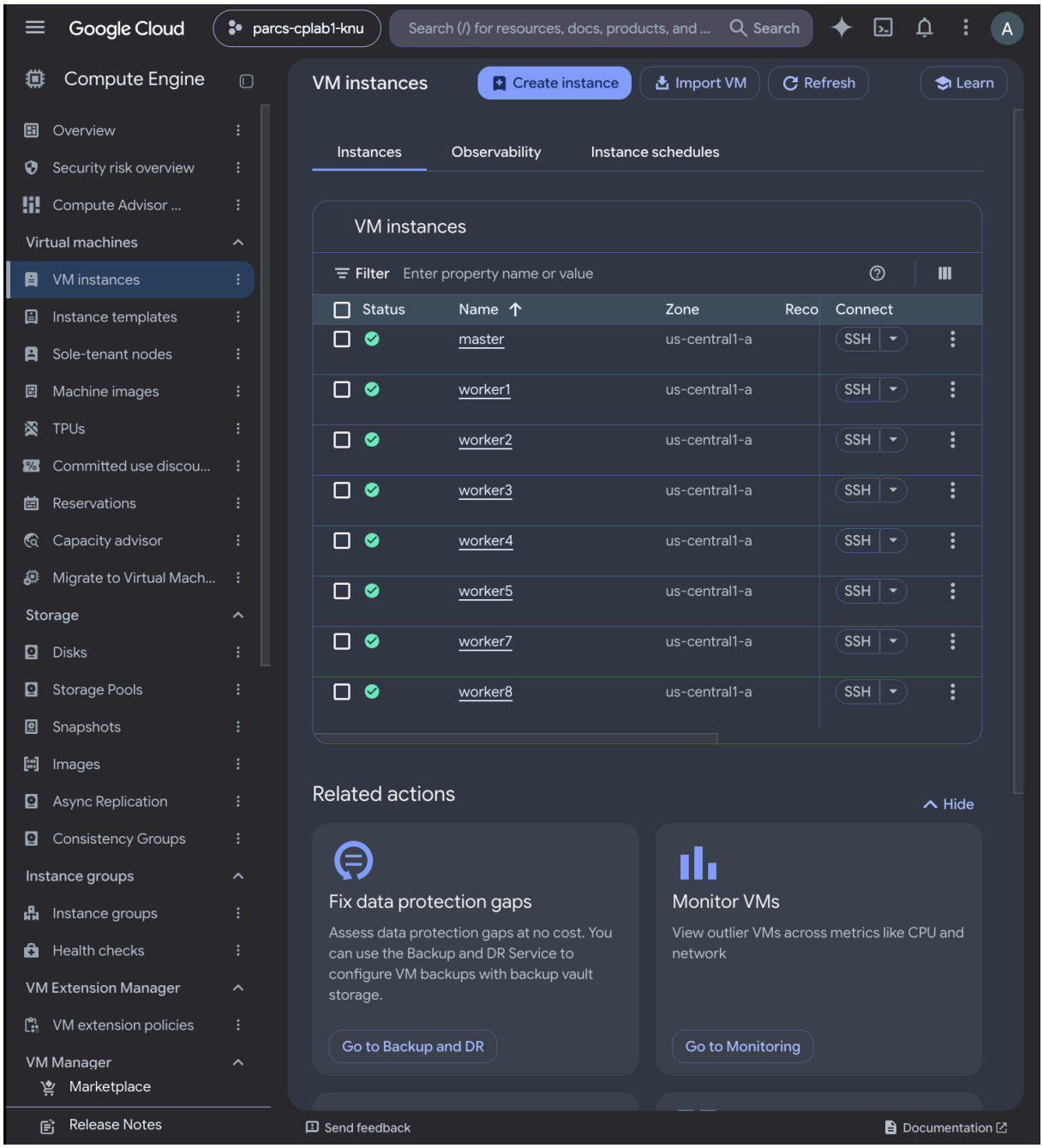


Рис. 1: Список VM у Google Compute Engine після розгортання

```
======================================================================
Benchmark: Решето Ератосфена — послідовне vs паралельне
======================================================================
Платформа: multiprocessing на VM з N vCPU.

──────────────────────────────────────────────────────────────────────
N = 10 000 000
──────────────────────────────────────────────────────────────────────
  Серіальний:               664579 простих, 1.081 с
  Воркерів     Час (с)      Speedup      Коректно
  1            1.369        0.79         ✓
  2            0.725        1.49         ✓
  4            0.382        2.83         ✓
  8            0.365        2.96         ✓

──────────────────────────────────────────────────────────────────────
N = 50 000 000
──────────────────────────────────────────────────────────────────────
  Серіальний:               3001134 простих, 5.373 с
  Воркерів     Час (с)      Speedup      Коректно
  1            7.512        0.72         ✓
  2            3.824        1.40         ✓
  4            1.971        2.73         ✓
  8            1.884        2.85         ✓

──────────────────────────────────────────────────────────────────────
N = 100 000 000
──────────────────────────────────────────────────────────────────────
  Серіальний:               5761455 простих, 10.937 с
  Воркерів     Час (с)      Speedup      Коректно
  1            14.434       0.76         ✓
  2            7.922        1.38         ✓
  4            4.064        2.69         ✓
  8            3.717        2.94         ✓

ahorobiichenko@cloudshell:~/CPLab1 (parcs-cplab1-knu)$
```

Рис. 2: Вивід `benchmark.py` на master VM (e2-standard-8, 8 vCPU)

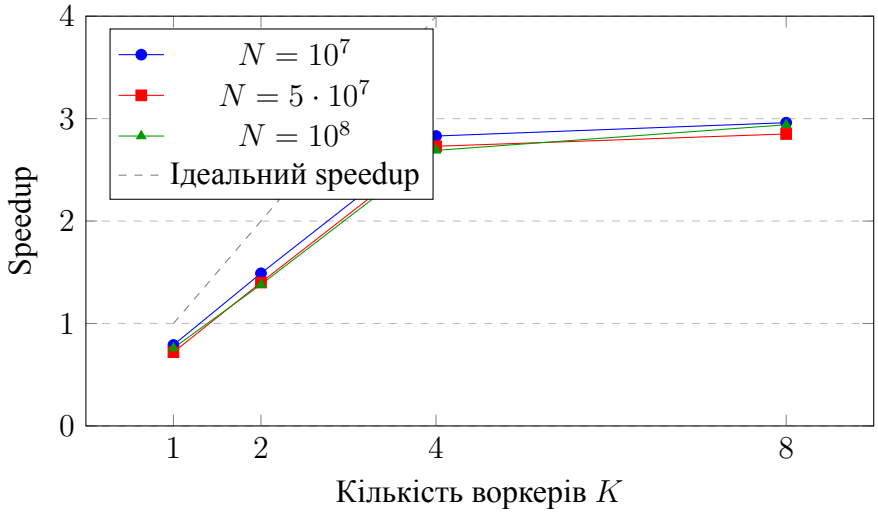


Рис. 3: Speedup паралельного алгоритму залежно від кількості воркерів